```
    if n == 0 or n == 1:
        return 1

    # Recursive Step: Function calls itself with smaller input
    else:
        # Example: factorial(4) returns 4 * factorial(3)
        return n * factorial(n - 1)

print(f"Factorial of 5: {factorial(5)}") # Output: 120
```

# Core Data Structures (Collections)

## 5.1 Strings (Immutable Sequence)

Strings are sequences of Unicode characters. They are **immutable**, meaning once created, their contents cannot be changed.

### Creating Strings

You can create strings using single, double, or triple quotes. Triple quotes are often used for multi-line strings or docstrings.

| Creation Method | Example | Real-time Use Case |
|---|---|---|
| **Single Quotes** | name = 'Alice' | Simple, single-line text data. |
| **Double Quotes** | message = "Hello, World!" | Preferred for consistency, allows easy use of single quotes inside (e.g., "It's time"). |
| **Triple Quotes** | html_content = """<html>...</html>""" | Storing multi-line text, like HTML templates, SQL queries, or poems. |

Python

```
# Real-time Example: Storing a user-provided comment
user_comment = "This is a great product!"  # Double quotes
license_agreement = '''
Please read the terms below.
By using this software, you agree to all conditions.
(c) 2024 TechCorp
''' # Triple quotes for multi-line text
print(f"Comment: {user_comment}")
print(license_agreement)
```

### Indexing (Positive and Negative)

You access individual characters in a string using their index (position).

- **Positive Indexing:** Starts from **0** for the first character and goes up to $n-1$ (where $n$ is the string length).
- **Negative Indexing:** Starts from **-1** for the last character and goes down to $-n$.

Python
```
# Real-time Example: Extracting the first and last initial of a user's ID
user_id = "PYTHON_DEV_2024"
first_char = user_id[0] # P (Positive index 0)
last_char = user_id[-1] # 4 (Negative index -1)

print(f"User ID: {user_id}")
print(f"First character: {first_char}")
print(f"Last character: {last_char}")
```

**Slicing $[start:stop:step]$**

Slicing extracts a substring. The slice includes the character at start but **excludes** the character at stop.

- **start:** Index where the slice begins (inclusive). Default is 0.
- **stop:** Index where the slice ends (exclusive). Default is end of string.
- **step:** The step size to take. Default is 1.

Python
```
# Real-time Example: Parsing a timestamp string "YYYY-MM-DD HH:MM:SS"
timestamp = "2025-12-12 18:00:25"

# Extract the Date part
date_part = timestamp[0:10] # or timestamp[:10] -> '2025-12-12'

# Extract the Time part
time_part = timestamp[11:] # -> '18:00:25'

# Extract the Year (using step of 2 to skip hyphen/separator)
# This example is slightly contrived but shows the step
year_with_sep = timestamp[0:4] # '2025'
every_other_char = timestamp[::2] # '20-1-1 8025' (Step 2)

print(f"Full Timestamp: {timestamp}")
print(f"Date: {date_part}")
print(f"Time: {time_part}")
```

**String Methods**

| Method | Description | Real-time Example |
|---|---|---|
| **len(s)** | Returns the length of the string (a built-in function). | Validating a password length: if len(password) < 8: |

| Method | Description | Real-time Example |
|---|---|---|
| **.lower()** | Converts all characters to lowercase. | Normalizing user input for a case-insensitive search: search_key.lower() |
| **.upper()** | Converts all characters to uppercase. | Formatting an abbreviation: country_code.upper() |
| **.strip()** | Removes leading and trailing whitespace. | Cleaning user form input: username.strip() |
| **.split(sep)** | Splits the string into a list of substrings using a delimiter (sep). | Parsing a CSV row: row_list = line.split(',') |
| **.join(iterable)** | Joins elements of an iterable (like a list) into a single string using the string as a separator. | Constructing a file path: '\\'.join(['C:','Users','Doc']) |
| **.find(sub)** | Returns the lowest index of substring sub found, or -1 if not found. | Checking if an email contains '@': email.find('@') |
| **.replace(old, new)** | Returns a new string with all occurrences of old replaced by new. | Sanitizing text by removing profanity: comment.replace('bad', '***') |

Python

```
# Real-time Example: Processing a Log Entry
log_entry = "  ERROR: Connection failed, retry 1. "

# 1. Clean up (strip) and convert to lowercase for analysis
cleaned_entry = log_entry.strip().lower()
print(f"1. Cleaned: '{cleaned_entry}'") # 'error: connection failed, retry 1.'

# 2. Check for the error keyword (find)
error_index = cleaned_entry.find("error")
print(f"2. Error found at index: {error_index}") # 0

# 3. Split the entry to separate the type and message
parts = cleaned_entry.split(': ')
log_type = parts[0]
log_message = parts[1]
print(f"3. Type: {log_type}, Message: {log_message}")

# 4. Join components for a new standardized format
standard_log = " | ".join([log_type.upper(), "SYSTEM_A", log_message.capitalize()])
print(f"4. Standard Log: {standard_log}") # 'ERROR | SYSTEM_A | Connection failed, retry 1.'
```

**String Formatting**

| Method | Syntax | Description |
|---|---|---|
| **% operator** | "%s %d" % ("text", 10) | Older C-style formatting. %s for string, %d for integer, %f for float. |
| **.format() method** | "{} {}".format("text", 10) | Uses curly braces as placeholders. Can use positional or keyword arguments. |
| **f-strings** | f"text {var}" | **Recommended method** (Python 3.6+). Prefix with f and embed variables/expressions directly in curly braces. |

Python
```
# Real-time Example: Generating a confirmation message
order_id = "TX-98765"
product_name = "Laptop"
price = 1250.75
tax_rate = 0.08

# Using .format()
msg_format = "Order ID: {}, Product: {}, Total: ${:.2f} (Includes {:.0%} Tax)".format(
   order_id, product_name, price * (1 + tax_rate), tax_rate
)

# Using f-strings (Recommended)
msg_fstring = f"Order ID: **{order_id}**. Product: {product_name}. Total: **${price * (1 + tax_rate):.2f}** (Includes {tax_rate:.0%} Tax)"

print(f"Format method: {msg_format}")
print(f"F-string method: {msg_fstring}")
```

**String Immutability**

Immutability means you **cannot** change a string in place. Any string method (like .lower(), .replace()) returns a **new string**; it does not modify the original.

Python
```
original_tag = "python"
modified_tag = original_tag.upper() # A new string is created

print(f"Original: {original_tag}")   # 'python'
print(f"Modified: {modified_tag}")   # 'PYTHON'

# Attempting to change an item will result in a TypeError
# original_tag[0] = 'P' # Uncommenting this line causes: TypeError: 'str' object does not support item assignment
```

---

## 5.2 Lists (Mutable Sequence)

Lists are ordered sequences of items, and they are **mutable**, meaning their contents can be changed after creation. Items in a list do not have to be of the same type.

### Creating and Accessing Lists

Lists are created using square brackets `[]`. Accessing uses indexing, just like strings.

Python
```
# Real-time Example: Tracking user scores in a game
user_scores = [150, 200, 180, 250, 150]
user_names = ["Priya", "Amit", "Chris", "Diya"]

# Accessing
print(f"Highest score (first element): {user_scores[0]}")
print(f"The second user: {user_names[1]}") # Amit
```

### List Indexing and Slicing

Identical to strings, lists support positive and negative indexing, and slicing `[start:stop:step]`.

Python
```
# Extracting a leaderboard's top 3
leaderboard = user_scores[0:3] # [150, 200, 180]
last_two = user_scores[-2:]   # [250, 150]
print(f"Leaderboard top 3 scores: {leaderboard}")
```

### List Methods

These methods *modify* the list in place (mutability) or return information about the list.

| Method | Description | Real-time Use Case |
|---|---|---|
| **.append(item)** | Adds a single item to the end of the list. | Adding a new task to a to-do list. |
| **.insert(index, item)** | Inserts an item at a specified index. | Inserting a high-priority item at the beginning of a queue. |
| **.extend(iterable)** | Appends all elements from an iterable (like another list) to the end. | Merging results from multiple database queries. |
| **.remove(item)** | Removes the first occurrence of a specific item by value. Raises `ValueError` if not found. | Removing a specific element from a list of user permissions. |
| **.pop(index)** | Removes and **returns** the item at a given index (defaults to the last item). | Dequeuing an item from a list used as a stack or queue. |

| Method | Description | Real-time Use Case |
|---|---|---|
| **.sort()** | Sorts the list elements in place (ascending by default). | Arranging search results by price or date. |
| **.reverse()** | Reverses the order of elements in place. | Displaying most recent log entries first. |

Python
```
# Real-time Example: Managing a Shopping Cart
shopping_cart = ["Apples", "Milk", "Bread"]
print(f"Initial Cart: {shopping_cart}") # ['Apples', 'Milk', 'Bread']

# 1. Add a new item (append)
shopping_cart.append("Eggs")
print(f"After Append: {shopping_cart}") # ['Apples', 'Milk', 'Bread', 'Eggs']

# 2. Add an item at a specific position (insert)
shopping_cart.insert(1, "Bananas") # Insert at index 1
print(f"After Insert: {shopping_cart}") # ['Apples', 'Bananas', 'Milk', 'Bread', 'Eggs']

# 3. Remove an item by value (remove)
shopping_cart.remove("Bread")
print(f"After Remove: {shopping_cart}") # ['Apples', 'Bananas', 'Milk', 'Eggs']

# 4. Remove the last item and process it (pop)
last_item = shopping_cart.pop() # Removes 'Eggs'
print(f"Removed item: {last_item}, Current Cart: {shopping_cart}")

# 5. Sort the cart alphabetically (sort)
shopping_cart.sort()
print(f"After Sort: {shopping_cart}") # ['Apples', 'Bananas', 'Milk']
```

### List Mutability and Copying (Shallow vs. Deep Copy)

Because lists are mutable, special care is needed when copying.

- **Assignment ($=$):** Just creates a new reference to the **same** list object. Changing one affects the other.
- **Shallow Copy (e.g., slicing `[:]` or `.copy()`):** Creates a **new** list object. If the list contains simple immutable objects (numbers, strings), it works like a deep copy. If it contains **mutable objects** (like other lists), only the *references* to those nested objects are copied. Changing a *nested* mutable object affects both lists.
- **Deep Copy (using `copy.deepcopy()`):** Creates a new list object and recursively copies all nested objects.

Python
```
import copy

# Example with simple (immutable) elements
list_a = [1, 2, 3]
list_b = list_a[:] # Shallow copy using slicing

list_b[0] = 99
print(f"List A: {list_a}, List B: {list_b}") # A: [1, 2, 3], B: [99, 2, 3] (Independent)
```

```
# Example with mutable (nested list) elements
original_list = [10, [20, 30], 40]

# Shallow Copy
shallow_copy = original_list.copy()
shallow_copy[1][0] = 999 # Modify the nested list [20, 30]

print(f"Original (Shallow): {original_list}") # [10, [999, 30], 40] <- MODIFIED!
print(f"Shallow Copy: {shallow_copy}")      # [10, [999, 30], 40]

# Deep Copy
deep_copy = copy.deepcopy(original_list)
deep_copy[1][0] = 111 # Modify the nested list

print(f"Original (Deep): {original_list}") # [10, [999, 30], 40] <- *STILL* MODIFIED from SHALLLOW copy above
print(f"Deep Copy: {deep_copy}")         # [10, [111, 30], 40]
# Note: To see deep copy success, you would run the deep copy code *before* the shallow copy change.
```

### List Comprehensions (Basic Syntax)

A concise way to create lists. Syntax: `[expression for item in iterable if condition]`.

Python
```
# Real-time Example: Filtering and transforming data
data = [10, 25, 42, 60, 75, 90]

# 1. Filter: Get only the scores > 50
high_scores = [score for score in data if score > 50] # [60, 75, 90]

# 2. Transform: Convert temperatures from Celsius to Fahrenheit (C * 9/5 + 32)
celsius_temps = [20, 25, 30, 35]
fahrenheit_temps = [temp * 9/5 + 32 for temp in celsius_temps] # [68.0, 77.0, 86.0, 95.0]

print(f"High Scores: {high_scores}")
print(f"Fahrenheit: {fahrenheit_temps}")
```

### Using Lists as Stacks and Queues (Brief)

- **Stack (LIFO: Last-In, First-Out):** Use **`.append()`** to push (add) and **`.pop()`** to pop (remove from the end).
- **Queue (FIFO: First-In, First-Out):** Use **`.append()`** to enqueue (add to the end) and **`.pop(0)`** to dequeue (remove from the beginning). *Note: `pop(0)` is slow for large lists. For a fast queue, use `collections.deque`.*

Python
```
# Stack Example: Managing browser history (Last visited page is the first to go back to)
history = ["Google", "Wikipedia", "Python Docs"]
history.append("StackOverflow") # Push
last_page = history.pop()      # Pop -> 'StackOverflow'

# Queue Example: Processing print jobs
print_queue = ["Job A", "Job B", "Job C"]
print_queue.append("Job D") # Enqueue
next_job = print_queue.pop(0) # Dequeue -> 'Job A'
```

---

## 5.3 Tuples (Immutable Sequence)